Cheaper Re-planning, not Less Re-planning: An Efficiency Study of SmolVLA on a Turing GPU

Minh-Ngoc Luong
Hanoi University of Science and Technology
ngoclm004@gmail.com

Abstract

Vision-language-action (VLA) policies that re-plan at every control step pay the full cost of the model at every step. This report studies where that cost goes, and which ways of reducing it preserve closed-loop success, for the released LIBERO checkpoint of SmolVLA on a Tesla T4. The released configuration needs 789 ms per re-plan, 24 times the simulator step. Two overheads dominate: bfloat16 arithmetic, which Turing GPUs emulate, and the 10-step flow-matching loop of the action expert, whose small kernels leave the GPU waiting on kernel launches. Methods are compared on identical episodes with common random numbers and paired tests. Re-planning less often fails: executing 5 actions per chunk, re-planning at predicted gripper switches, or reusing a 5-step-old image all lower success by 12.0–12.5 points (Holm-adjusted $p \leq 0.012$). Re-planning more cheaply succeeds: float16 weights, 3 Euler steps and a CUDA-graph replay of the flow loop cut a re-plan to 112 ms (7.0×); the graph is bit-exact, and success is unchanged on 200 paired episodes (77.0% vs 77.0%). ***[500-pair non-inferiority result.]*** *The remaining 84 ms is the vision-language prefix, which fixes the next target. Code: https://github.com/Ngoc-LM/efficient-vla.*

1. Introduction

VLA policies map camera images, a language instruction and the robot state to actions with a pretrained vision-language model (VLM) and an action head [1, 7, 18]. Their cost is paid inside a control loop: at batch size one, every re-plan waits for the full model before the robot can act. Work on efficient VLAs reduces this cost in several places: smaller backbones [18, 21], fewer tokens or layers per step [22–24], cheaper action decoding [8, 16], asynchronous execution [2, 17], and inference systems [6, 14]. Whether a given saving preserves closed-loop behaviour depends on the checkpoint and the hardware: exporting the SmolVLA checkpoint used here to ONNX cut its median latency by 40% but, through truncated instructions, also cut LIBERO-Spatial success from 70% to 41% [6].

This report examines SmolVLA [18], a 0.45B-parameter flow-matching VLA designed for affordable hardware, on the LIBERO-Spatial benchmark [13] and a Tesla T4, a GPU without bfloat16 tensor cores. The scope is training-free: weights, architecture and benchmark protocol stay fixed, and only how the policy is executed changes. The report covers the four parts of the assignment:

- a study of efficient VLAs and of diffusion versus flow-matching action generation, with a literature matrix of 18 verified sources (Sec. 2);
- a reproduction of the released checkpoint with LeRobot, including the differences from the paper configuration (Sec. 3);
- an analysis of strengths, limitations, trajectories and computational bottlenecks (Sec. 4);
- an improvement study that tests two hypotheses on identical episodes: re-planning less often, and re-planning more cheaply (Sec. 5).

The individual levers that make re-planning cheaper (half precision, fewer flow steps, CUDA graphs) are known [14, 20]; the contribution is a measured account of why each one helps on this hardware and a paired closed-loop test of whether the policy still succeeds.

2. Efficient VLAs and Action Generation

Where efficient VLAs save cost. Tab. 1 groups the efficiency work in the literature matrix (18 sources, each read from the paper itself; delivered as a spreadsheet) by the part of the inference loop it changes. Three assumptions recur. Token and layer reduction assumes redundancy: consecutive frames share most visual content [22], or easy situations need fewer layers [24]; both save compute only in the language model, not in the vision encoder. Chunked and asynchronous execution assumes that actions predicted from an older observation remain valid; SmolVLA's asynchronous mode shortened task completion by 30% but lowered success from 78.3% to 73.3% [18], and A2C2 adds a trained

Lever	Examples	Reported effect (hardware)
Smaller model	SmolVLA [18], TinyVLA [21]	0.45B params, LIBERO avg 87.3; 14 vs 292 ms per action against OpenVLA (A6000)
Fewer tokens or layers	VLA-Cache [22], EfficientVLA [23], DeeR [24]	1.63× (OpenVLA, LIBERO, 4090); 1.93× at 28.9% FLOPs, −0.6 SR (A40); LLM FLOPs 5.2–6.5× lower (trained)
Action decoding	FAST [16], OFT [8], CogACT [11]	up to 5× faster training but 750 vs 100 ms per chunk; 26× throughput; VLM runs once, small DiT denoises
Execution	RTC [2], A2C2 [17]	tolerates >300 ms delay at +21 ms per chunk; 4.7 ms head per step
Systems	Realtime-VLA [14], ONNX [6], Coda [20]	π_0 106.5→27.3 ms (4090); 40% lower median latency with an SR drop; 2-step flow with a learned corrector

Table 1. Efficiency levers in the literature matrix (selection). Numbers as reported by each paper.

correction head to restore reactivity [17]. Systems work assumes that the model stays numerically the same, which an export can violate [6]. Two open questions follow for this study. First, speed-ups are usually reported next to aggregate success rates, yet a non-significant difference is not evidence of equivalence [6]; this study therefore fixes a non-inferiority margin in advance and tests it on paired episodes. Second, most latencies are measured on GPUs with native bfloat16 support (RTX 4090, A100, A40), so it is unclear which bottleneck dominates on older or embedded GPUs.

Diffusion versus flow matching. Both families learn to transform Gaussian noise $\boldsymbol{\epsilon}$ into an action chunk $\mathbf{A}$ conditioned on the observation $\mathbf{o}$. Diffusion heads (Diffusion Policy [3], CogACT [11]) use the DDPM forward process $\mathbf{A}_k = \sqrt{\bar{\alpha}_k}\,\mathbf{A} + \sqrt{1-\bar{\alpha}_k}\,\boldsymbol{\epsilon}$ and train a network to predict the injected noise [4],

$$\mathcal{L}_{\text{DDPM}} = \mathbb{E}_{k,\boldsymbol{\epsilon}} \big\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\mathbf{A}_k, k \mid \mathbf{o}) \big\|^2. \tag{1}$$

Flow-matching heads (π_0 [1], SmolVLA) interpolate linearly between noise and data and regress the constant velocity of that path [12] (Eq. (2)). The two differ in four respects. (i) *Target*: noise versus velocity. (ii) *Path*: the linear interpolation gives each training pair a constant velocity, which yields straighter sampling trajectories than diffusion paths and favours coarse ODE steps [12]; on RoboTwin, Coda reports $\pi_{0.5}$ at 71.3% with 3 flow steps against 71.5% with 10, and 59.8% with one [20]. (iii) *Sampler*: Diffusion Policy samples with 100 DDPM steps in simulation and with 10 DDIM [19] steps on the real robot, CogACT with 10 DDIM steps and classifier-free guidance, while flow models integrate the ODE with 10 forward-Euler steps. (iv) *Architecture*: Diffusion Policy repeats its whole denoising network (9–80M parameters) on visual features computed once; CogACT, π_0 and SmolVLA run a large VLM once and repeat only a head (a DiT conditioned on one cognition feature, or an action expert that attends to the cached VLM keys and values). In the last three, the cost of one step is set by the size of the head and, at batch size one, by kernel-launch overhead, which Sec. 4 measures.

3. Reproduction

Model and checkpoint. The study uses the released LeRobot implementation of SmolVLA [18] (v0.6.1) and the LIBERO-finetuned checkpoint `HuggingFaceVLA/smolvla_libero`. The checkpoint is not the configuration reported in the paper: it keeps all 32 decoder layers of the SmolVLM2-500M backbone instead of the first 16, uses an action expert of width $0.5\times$ instead of $0.75\times$, and re-plans at every control step (n_{exec}=1 executed action per predicted chunk). It has 605M parameters (86M vision encoder, 12M connector, 362M language model, 97M action expert), stored in bfloat16.

Pipeline. At control step k the policy receives two RGB images (third-person and wrist cameras, rotated by 180° and resized with padding to 512×512), the task instruction (at most 48 tokens), and an 8-D proprioceptive state $\mathbf{s}_k$ (end-effector position, axis-angle orientation, two gripper joints). SigLIP encodes each image, a pixel-shuffle connector compresses it to a short token sequence, and the prefix [image tokens; language tokens; state token] is processed once by the language model, whose per-layer keys and values are cached. The action expert is a smaller transformer that predicts a chunk of H=50 actions $\mathbf{A} \in \mathbb{R}^{H\times 7}$ (end-effector delta pose and gripper command); its layers alternate cross-attention to the cached prefix and self-attention over action tokens.

Training objective. SmolVLA is trained with conditional flow matching on a linear path between data and noise. With $\boldsymbol{\epsilon} \sim \mathcal{N}(0, I)$, $\tau \sim \text{Beta}(1.5, 1)$ rescaled to $[0.001, 1]$, and $\mathbf{A}^\tau = \tau\boldsymbol{\epsilon} + (1-\tau)\mathbf{A}$, the expert v_θ regresses the constant velocity of the path,

$$\mathcal{L}(\theta) = \mathbb{E}_{\tau,\boldsymbol{\epsilon}} \big\| v_\theta(\mathbf{A}^\tau, \tau \mid \mathbf{o}) - (\boldsymbol{\epsilon} - \mathbf{A}) \big\|_2^2. \tag{2}$$

Action generation. At inference the chunk is obtained by integrating $d\mathbf{A}/d\tau = v_\theta$ from τ=1 (noise) to τ=0 with S forward-Euler steps, $\mathbf{A} \leftarrow \mathbf{A} - \frac{1}{S}\, v_\theta(\mathbf{A}, \tau \mid \mathbf{o})$; the released configuration uses S=10. Each re-plan therefore costs one prefix pass (vision, language model) and S expert passes that reuse the prefix cache. The first n_{exec} actions are unnormalised and executed by the operational-space controller at 20 Hz.

Evaluation metric. Success rate (SR): the fraction of episodes in which the goal predicate of the task (*e.g.* `On(bowl, plate)`) holds before the step limit (280 steps on LIBERO-Spatial). Each episode is fixed by a (task, stored

Item	Setting
Checkpoint	`HuggingFaceVLA/smolvla_libero`
Code	LeRobot v0.6.1 (`7e241bd`), PyTorch 2.11
Simulator	LIBERO (hf-libero 0.1.4), MuJoCo 3.3.7, EGL
Hardware	Kaggle, 2× Tesla T4 (16 GB), 4 vCPU
Precision	bfloat16 backbone (released); float16 (ours)
Episodes	10 tasks × 20 (or 50) stored initial states
Flow steps S	10 (released); 3, 2, 1 (ours)
Re-planning	every step, n_{exec}=1 (released)

Table 2. Reproduction setup.

initial state) pair. The 10 LIBERO-Spatial tasks are evaluated on initial states 0–19 (200 episodes; 0–49 for the main comparison), and methods are compared *paired* on identical episodes, with common random numbers: the flow noise at control step k of an episode is seeded by (episode, k) for every method.

Setup and differences from the original work. Tab. 2 lists the setup. Three differences matter. (i) The checkpoint configuration above differs from the paper. (ii) MuJoCo is pinned to 3.3.7: from 3.4.0 the stored initial states of task 5 settle differently and SmolVLA drops from 80% to 28% on that task [10]. (iii) The evaluation uses 20 episodes per task instead of 10, a T4 GPU, and the LeRobot evaluation loop. The reproduced SR of 77.0% [70.7, 82.3] (Wilson 95% CI) is below the 90% reported for the paper configuration [18] but inside the range obtained by others with this checkpoint (70–83%) [6, 9]; A2C2 reports the same 77% for a SmolVLA trained from scratch on LIBERO with n_{exec}=1 [17].

4. Analysis

Computational bottlenecks. Tab. 3 breaks down one re-plan on a T4 (batch 1, CUDA events, warm-up, interleaved rounds, median of 60 calls). Three observations follow. (i) The simulator is not the bottleneck: one LIBERO step (physics and rendering of two cameras) takes 32 ms, against 814 ms for the policy. (ii) The vision encoder costs 230 ms in the released bfloat16 weights but 39 ms in float16 with identical weights: Turing GPUs have no bfloat16 tensor cores, so bfloat16 matrix products are emulated. (iii) The action expert accounts for 18% of the FLOPs of a re-plan but 63% of its latency (10 × 51 ms), and its per-step time is the same in bfloat16 and float16. A denoising pass is small (50 action tokens, 11.5 GFLOP) but launches several hundred kernels; at batch size 1 the GPU waits on kernel launches rather than on arithmetic.

Successful and failed trajectories. Successful episodes finish in a median of 100 steps; every failure runs to the 280-step limit. All 46 failures contain at least one grasp attempt. From the executed gripper commands (a proxy, since object

Component (one re-plan)	GFLOP	bf16 (ms)	fp16 (ms)
Vision encoder, 2 cameras	431	230	39
Prefix incl. vision (32 LM layers)	527	304	82
One expert pass (flow step)	11.5	51	52
Re-plan, S=10 (released)	642	814	605
Re-plan, S=3	562	458	243
Simulator step (CPU + EGL)	–	32	

Table 3. Cost of one re-plan on a Tesla T4 (median latency of isolated components, one profiling run; the end-to-end re-plan times of Tab. 4, measured in a separate run, agree within 3%).

poses are not logged), 28 failures release the bowl away from the goal after one or two grasps, 9 end while still holding the bowl, and 9 repeat the grasp three or more times. Failures are uneven across tasks: task 3 (bowl on the cookie box) fails in 10 of its 20 episodes, tasks 1 and 5 in one each. Re-planning at every step with fresh flow noise makes consecutive chunks differ about as much as independent samples. Measured on 581 control steps of LIBERO-Object (a suite used only for calibration), at a fixed observation two 10-step chunks with different noise differ by 1.37 (L2 norm of the first action, normalised units), and the previous chunk shifted by one step differs from the current one by 1.31. The gripper command chatters accordingly: failed episodes contain 12.6 gripper segments of at most 3 steps per 100 steps against 2.7 in successes, and every failure has at least three such segments.

Strengths. S1 Small footprint. 605M parameters, 1.16 GB of weights and 21 MB of activations per re-plan in float16 fit a single 16 GB T4 with room for several processes. **S2 Prefix caching.** The language model processes the observation once per re-plan and the expert reads its cached keys and values, so an additional flow step costs 52 ms against 304 ms for the prefix. **S3 Few-step flow.** The linear flow path makes the integration robust to coarse discretisation: with S=3 the first action deviates from the S=10 result by 0.32 (normalised L2, LIBERO-Object), four times less than the sampling spread of the policy (1.37), and SR on LIBERO-Spatial does not drop (Sec. 5).

Limitations. L1 Precision mismatch on older GPUs. bfloat16 weights run emulated on Turing; the vision encoder is 5.9× slower than in float16 (Tab. 3). **L2 Launch-bound expert.** The expert loop uses a small fraction of the GPU and dominates latency at batch 1. **L3 Dependence on fresh visual input.** This checkpoint needs a new image at every step: executing 5 actions per chunk loses 12.5 points of SR, and re-planning every step with fresh proprioception but a 5-step-old image loses 12.0 points (Sec. 5). Compute cannot be amortised across steps, so an episode needs about 150 full re-plans. **L4 Temporally inconsistent actions.** Independent noise at every re-plan produces gripper chatter, which is

concentrated in failures.

5. Improvement: Cheaper, not Less, Re-planning

Bottleneck and hypotheses. The released checkpoint re-plans at every control step, and a re-plan costs 789 ms (Tab. 4); a successful episode (median 100 steps) therefore spends about 79 s in the policy and 3 s in the simulator. Two hypotheses address this cost. **H1 (re-plan less):** computation can be shared across control steps without lowering SR, by executing several actions of a chunk, by re-planning only when the predicted gripper command switches, or by reusing the visual prefix of an older image with fresh proprioception. **H2 (re-plan more cheaply):** each re-plan can be made cheaper while the policy computes the same function up to precision and discretisation: float16 removes the emulated bfloat16 arithmetic, fewer Euler steps remove discretisation slack (with $S{=}3$ the first action deviates by 0.32 from $S{=}10$, against a sampling spread of 1.37; Sec. 4), and a CUDA graph removes the launch overhead of the expert loop. H2 predicts SR within the non-inferiority margin and, because launch overhead hides arithmetic, a larger graph speed-up once precision is fixed.

Implementation. Open-loop execution sets the number of executed actions per chunk to k; the event trigger additionally discards the queue when the next queued action switches the gripper sign (then waits 5 steps). The stale-image variant caches the keys and values of the image and language tokens, recomputes them every 5 steps or at a predicted gripper switch, and appends the current state token at every step. Float16 casts the VLM and expert weights (the same parameters) and keeps the projections in float32. For the CUDA graph, the prefix runs eagerly and the S-step Euler loop of the expert is captured once per S with static buffers for the noise, the prefix cache and the time steps, then replayed at every re-plan; capture takes 0.25–2.2 s once, and the replayed actions equal the eager ones exactly (maximum absolute difference 0 in all 8 settings).

Protocol. All variants run the 10 LIBERO-Spatial tasks on stored initial states 0–19 (200 episodes; 0–49 for the main comparison) with common random numbers. Differences in SR are paired: exact McNemar test, Newcombe 95% confidence interval [15], and Holm adjustment [5] over the comparisons with the released policy. A variant is non-inferior if the lower confidence bound exceeds -5 points, a margin fixed before the runs. Latencies are measured on isolated calls (Tab. 4), because the closed-loop runs share each GPU between two processes.

	bfloat16 (released)		float16	
Euler steps S	eager	graph	eager	graph
10 (released)	789	572	616	178
3	433	372	245	**112**
2	386	345	191	103
1	335	318	136	93
Fit $a{+}bS$	$284{+}50.5S$	$289{+}28.3S$	$84{+}53.3S$	$84{+}9.4S$

Table 4. Latency of one re-plan (ms, median of 60 calls, Tesla T4, batch 1). The fit is least squares over the four values of S (residuals $\leq$2.7 ms).

Variant	Vis./100	SR	ΔSR [95% CI]	p_{Holm}
Released (bf16, $S{=}10$)	100	77.0	–	–
H1: re-plan less				
Open loop, $k{=}5$	20.1	64.5	−12.5 [−20.3, −4.5]	0.012
Open loop, $k{=}10$	10.2	67.5	−9.5 [−17.3, −1.6]	0.074
$k{=}5$ + gripper events	21.5	65.0	−12.0 [−18.7, −5.2]	0.006
Stale image ($k{=}5$, $S{=}3$)	24.6	65.0	−12.0 [−19.1, −4.7]	0.009
H2: re-plan more cheaply				
bf16, $S{=}3$	100	79.0	+2.0 [−3.8, +7.8]	1.000
fp16, $S{=}3$	100	77.0	0.0 [−5.8, +5.8]	1.000
fp16, $S{=}2$	100	[–]	[–]	[–]
fp16, $S{=}1$	100	[–]	[–]	[–]

Table 5. Closed-loop results on 200 paired LIBERO-Spatial episodes. Vis./100: vision-encoder passes per 100 control steps.

Re-planning less lowers success (H1 rejected). The open-loop and event-triggered variants call the policy about five times less often but succeed in 12.0–12.5 fewer episodes per 100 (Tab. 5); with $k{=}10$ the drop of 9.5 points is not significant after Holm adjustment. The event trigger removes most gripper chatter (1.2 short gripper segments per 100 steps against 5.0) without recovering success. The stale-image variant keeps the expert and fresh proprioception at every step and still loses 12.0 points, so proprioception does not substitute for a current image. This checkpoint was fine-tuned to execute one action per re-plan; the benefit of longer execution horizons reported for a SmolVLA trained from scratch on LIBERO [17] does not transfer to it.

Re-planning more cheaply preserves success (H2). Along the path released $\rightarrow$ float16 $\rightarrow$ $S{=}3$ $\rightarrow$ graph, a re-plan takes 789, 616, 245 and 112 ms: factors of 1.28, 2.51 and 2.19, or 7.05$\times$ in total (Tab. 4). The linear fits explain the order of these factors. Without the graph, one Euler step costs 50.5 ms in bfloat16 and 53.3 ms in float16, so its cost does not depend on the arithmetic: the expert is launch-bound. With the graph, the step costs 28.3 ms in bfloat16 and 9.4 ms in float16; once launches are removed, the emulated arithmetic becomes visible, and the graph gains 1.38$\times$ in bfloat16 but 3.46$\times$ in float16 at $S{=}10$. On 200 paired episodes, bfloat16 with $S{=}3$ succeeds in 79.0% and float16 with $S{=}3$ in 77.0%, and neither differs significantly

from the released 77.0%; for float16 the lower confidence bound (-5.8) does not yet clear the margin. **[500 pairs: non-inferiority decision. S-curve: SR at S=2, 1. Fixed noise: SR and chatter vs fp16 S=3. In-loop latency with one process per GPU and in-loop equivalence of the graph.]**

Trade-offs and the next bottleneck. The float16 gain is specific to GPUs without bfloat16 tensor cores; on newer GPUs the bfloat16 arithmetic is native, and only the step and graph gains would apply (not measured here). Float16 changes the numerics slightly: at S=10 and fixed noise, actions differ from bfloat16 by 0.010 on average and 0.049 at most, against a mean action magnitude of 0.17, with no overflow observed. The graph requires static shapes (batch 1, fixed prefix and chunk length) and one capture per S. The optimised policy is still not real time: 112 ms per re-plan plus 32 ms of simulation gives about 7 Hz, against the 20 Hz control rate. The fit $T(S) \approx 84 + 9.4\,S$ ms shows where the remaining time is: the prefix takes 84 ms (75% at S=3; vision encoder 39 ms), so even S=1 would only reach 93 ms. Further gains require a cheaper vision-language prefix, which, unlike the levers above, would change the model and need training.

6. Conclusion

For the released LIBERO checkpoint of SmolVLA on a Tesla T4, the cost of a re-plan is set less by FLOPs than by two execution overheads: emulated bfloat16 arithmetic and the launch-bound flow loop of the action expert. Sharing computation across control steps lowers success by about 12 points, whereas removing the two overheads and the discretisation slack reduces a re-plan from 789 to 112 ms with bit-exact graph replay and unchanged success on 200 paired episodes. **[Update with the 500-pair decision and the S-curve.]**

Limitations. The study covers one checkpoint, one LIBERO suite and one GPU type, in simulation. The reproduced baseline (77.0%) is below the 90% reported for the paper configuration of SmolVLA, which differs from the released checkpoint (Sec. 3). The precision gain does not carry over to GPUs with bfloat16 tensor cores, and the step and graph gains were not measured on other hardware.

Future work. The remaining latency lies in the vision-language prefix (84 of 112 ms). Reducing it requires changing the model, for example distilling the policy into a shallower language model or a cheaper vision encoder, followed by the same paired closed-loop test. Measurements on an embedded GPU and on exported runtimes (ONNX, TensorRT) should use the same test, since a faster runtime can change closed-loop behaviour [6].

References

[1] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, et al. π_0: A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024. 1, 2

[2] Kevin Black, Manuel Y Galliker, and Sergey Levine. Real-time execution of action chunking flow policies. *arXiv preprint arXiv:2506.07339*, 2025. 1, 2

[3] Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin Burchfiel, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Robotics: Science and Systems*, 2023. 2

[4] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems*, pages 6840–6851, 2020. 2

[5] Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979. 4

[6] Rafiqul Islam. When faster vla deployment changes closed-loop behavior: Task success-latency analysis of smolvla across pytorch and onnx variants. *arXiv preprint arXiv:2609.14146*, 2026. 1, 2, 3, 5

[7] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, et al. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024. 1

[8] Moo Jin Kim, Chelsea Finn, and Percy Liang. Fine-tuning vision-language-action models: Optimizing speed and success. *arXiv preprint arXiv:2502.19645*, 2025. 1, 2

[9] LeRobot contributors. Cannot reproduce smolvla results on libero benchmark. GitHub issue huggingface/lerobot #2354, https://github.com/huggingface/lerobot/issues/2354, . Accessed October 2026. 3

[10] LeRobot contributors. mujoco>=3.4.0 silently breaks the initial-state premise of libero_spatial task 5. GitHub issue huggingface/lerobot #4390, https://github.com/huggingface/lerobot/issues/4390, . Accessed October 2026. 3

[11] Qixiu Li, Yaobo Liang, Zeyu Wang, Lin Luo, Xi Chen, Mozheng Liao, Fangyun Wei, Yu Deng, Sicheng Xu, Yizhong Zhang, et al. Cogact: A foundational vision-language-action model for synergizing cognition and action in robotic manipulation. *arXiv preprint arXiv:2411.19650*, 2024. 2

[12] Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In *International Conference on Learning Representations*, 2023. 2

[13] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. Libero: Benchmarking knowledge transfer for lifelong robot learning. In *Advances in Neural Information Processing Systems*, pages 44776–44791, 2023. 1

[14] Yunchao Ma, Yizhuang Zhou, Yunhuan Yang, Tiancai Wang, and Haoqiang Fan. Running vlas at real-time speed. *arXiv preprint arXiv:2510.26742*, 2025. 1, 2

[15] Robert G Newcombe. Improved confidence intervals for the difference between binomial proportions based on paired data. *Statistics in Medicine*, 17(22):2635–2650, 1998. 4

[16] Karl Pertsch, Kyle Stachowicz, Brian Ichter, Danny Driess, Suraj Nair, Quan Vuong, Oier Mees, Chelsea Finn, and Sergey Levine. Fast: Efficient action tokenization for vision-language-action models. *arXiv preprint arXiv:2501.09747*, 2025. 1, 2

[17] Kohei Sendai et al. Leave no observation behind: Real-time correction for vla action chunks. *arXiv preprint arXiv:2509.23224*, 2025. 1, 2, 3, 4

[18] Mustafa Shukor, Dana Aubakirova, Francesco Capuano, Pepijn Kooijmans, Steven Palma, Adil Zouitine, Michel Aractingi, Caroline Pascal, Martino Russi, Andres Marafioti, et al. Smolvla: A vision-language-action model for affordable and efficient robotics. *arXiv preprint arXiv:2506.01844*, 2025. 1, 2, 3

[19] Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models. In *International Conference on Learning Representations*, 2021. 2

[20] Zhipeng Tang, Xinda Chen, Weining Rao, Xiao Li, Wenting Tan, Yuning Wang, Xiao Shi, and Xiaofang Zhao. Fewer steps, better actions: Rethinking flow-matching inference for vla policies. *arXiv preprint arXiv:2609.21216*, 2026. 1, 2

[21] Junjie Wen, Yichen Zhu, Jinming Li, Minjie Zhu, Zhibin Tang, Kun Wu, Zhiyuan Xu, Ning Liu, Ran Cheng, Chaomin Shen, et al. Tinyvla: Towards fast, data-efficient vision-language-action models for robotic manipulation. *IEEE Robotics and Automation Letters*, 2025. 1, 2

[22] Siyu Xu, Yunke Wang, Chenghao Xia, Dihao Zhu, Tao Huang, and Chang Xu. Vla-cache: Towards efficient vision-language-action model via adaptive token caching in robotic manipulation. *arXiv preprint arXiv:2502.02175*, 2025. 1, 2

[23] Yantai Yang et al. Efficientvla: Training-free acceleration and compression for vision-language-action models. *arXiv preprint arXiv:2506.10100*, 2025. 2

[24] Yang Yue, Yulin Wang, Bingyi Kang, Yizeng Han, Shenzhi Wang, Shiji Song, Jiashi Feng, and Gao Huang. Deer-vla: Dynamic inference of multimodal large language models for efficient robot execution. In *Advances in Neural Information Processing Systems*, 2024. 1, 2

A. Reproducibility

Environment. `scripts/kaggle_setup.sh` installs LeRobot v0.6.1 with the `smolvla` and `libero` extras, pins MuJoCo below 3.4, registers the NVIDIA EGL vendor library for headless rendering, and answers LIBERO's first-import prompt. Versions: Python 3.13, PyTorch 2.11 (CUDA 12.8), transformers 5.5.4, hf-libero 0.1.4, MuJoCo 3.3.7.

Evaluation. `python -m evla.run_eval` runs one configuration on a list of (task, initial state) episodes and appends one CSV row per episode, so that interrupted runs resume and runs from several sessions can be concatenated. Each episode uses the environment seed $1000 + e$ with $e = 1000 \cdot \text{task} + \text{init}$, and the flow noise of the re-plan at control step k is drawn from a generator seeded with $10^4 e + k$ for every method (common random numbers). `scripts/run_queue.sh` runs a list of configurations as four processes on the two GPUs and pushes partial results every 20 minutes. `python -m evla.analyze` produces the paired comparisons, `python -m evla.bench_graph` the latency table, and `python -m evla.profile` the component breakdown.

Variants. Tab. 6 lists the command-line options of every evaluated variant.

Variant	`run_eval` options
Released	`--method baseline`
Open loop	`--method open_loop --k 5` (or 10)
Gripper events	`... --event-trigger --hold 5`
Stale image	`--method stale --k 5 --flow-steps 3`
Fewer steps	`--method flow --flow-steps 3`
Float16	`... --dtype fp16`
CUDA graph	`... --graph`
Fixed noise	`... --fixed-noise`

Table 6. Evaluated variants.

B. Additional Results

Per-task success. Tab. 7 gives the number of successful episodes out of 20 per task for each variant on initial states 0–19.

Failure categories. Failures of the released policy are categorised from the executed gripper commands, since object poses are not logged: a grasp is a switch from open to closed. *Released off target*: the episode ends with the gripper open after one or two grasps (28 of 46). *Holding at timeout*: the gripper is closed at the step limit (9). *Repeated grasps*: three or more grasps (9).

Task	0	1	2	3	4	5	6	7	8	9
Released	14	19	13	10	16	19	15	16	17	15
bf16, S=3	15	18	15	14	15	18	14	15	19	15
fp16, S=3	15	18	17	15	15	17	14	12	17	14
Open loop k=5	12	14	13	11	13	13	12	11	16	14
k=5 + events	12	16	10	12	13	17	12	12	15	11
Stale image	11	13	17	12	13	16	11	13	12	12
Open loop k=10	15	15	12	9	12	15	17	11	14	15

Table 7. Successful episodes out of 20 per LIBERO-Spatial task.

Warm-started flow (rejected before closed-loop tests). Starting the integration from the previous chunk, shifted by one step and re-noised to flow time t_0, was calibrated offline on 586 control steps of five LIBERO-Object episodes. The deviation of the first action from a 10-step integration with the same noise was 0.83, 0.53 and 0.34 for $t_0 = 0.3, 0.5, 0.7$ (3 steps), against 0.32 for a cold start with 3 steps and 0.51 with 2 steps; the shifted previous chunk alone deviated by 1.31, and a 10-step integration with different noise by 1.37. A warm start was therefore no more accurate than a cold start with the same number of steps and was not evaluated in closed loop.